

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 632 — Accessible Computing

Design for the constraint, and the design gets better for everybody

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 11 (with CSCE 671 and CSCE 679)

Prerequisites: CSCE 671 Human-Computer Interaction, particularly its Modules 02 and 09; CSCE 679 Data Visualization for Module 10; any course in which you built an interface somebody else used

Effort: 11–13 hours per week · 13 modules + 2 project weeks

Deliverable: An accessibility repair of something you built earlier — audited against the standard, tested with the actual assistive technology, with the automated tools' coverage measured rather than assumed, and the remaining defects listed

Course description

This course takes a position in Module 01 and spends twelve modules making it operational. The position is that a disability is a mismatch between a body and an environment, and that software is part of the environment — so an interface a blind user cannot operate is a defect in the interface, which is a statement about engineering rather than about accommodation.

The first half is the specific requirements, by capability rather than by diagnosis. Vision, hearing, motor control, and cognition each impose concrete constraints (Modules 03 through 06) — and each constraint has a test you can run in minutes, which is what makes this checkable rather than aspirational.

The second theme is that the standard is a floor and not a design. Module 07 covers WCAG properly, including what a conformance claim does and does not establish — because a page can satisfy every automatically checkable criterion and still be unusable, and Module 09 measures how large that gap actually is.

The third is that the mechanism matters more than the checklist. Module 08 is about the accessibility tree and the name-role-value model — because once you understand what an assistive technology actually receives, most of the rules become derivable rather than memorised, which is the same move CSCE 671 Module 05 makes.

And the closing argument is the one that makes this a design course. Designing for a constraint improves the design for everybody — captions are used most by people who can hear, and keyboard access is what power users want — so Module 13's honest claim is a quality claim, and the accessibility argument does not need charity to work.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 State the social and medical models and what follows from each.
- 2 Describe the range of disability, including temporary and situational.
- 3 State and test the requirements that vision imposes.
- 4 State and test the requirements for hearing and speech.
- 5 State and test the requirements for motor control.
- 6 State the cognitive and attentional requirements.
- 7 Explain WCAG's structure and what conformance establishes.
- 8 Explain the accessibility tree and the name-role-value model.
- 9 Test accessibility, and state what each method catches.
- 10 Make data and visual representations non-visually available.
- 11 Address accessibility in games and real-time graphics.
- 12 Explain why accessibility fails organisationally.
- 13 Claim accessibility honestly, and say what you tested.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Disability as a Design Failure	Who is the defect in?
2	The Range	How many people is this, really?
3	Vision	What does your interface sound like?
4	Hearing and Speech	What does your interface say that it does not also write?
5	Motor Control	Can you use this without a mouse?
6	Cognition, Attention, and Language	How much does this ask you to hold in your head?
7	The Standard	What does 'AA conformant' actually tell you?
8	Semantics and the Accessibility Tree	What does the assistive technology actually receive?
9	Testing	Your checker says zero errors. What does that mean?
10	Non-Visual Data	What does a scatterplot sound like?

Wk	Module	Core question it answers
11	Games and Real-Time Systems	Can your game be finished by somebody who cannot do the hard part?
12	Why It Fails	Everybody agrees it matters. Why is almost nothing accessible?
13	Claiming Something Is Accessible	'It's accessible.' Tested with whom, against what?

Projects

Project 1 — Audit something you built (after Module 6)

Take your own earlier interface and find out what it excludes.

Requirements

- **Something you built and shipped or submitted earlier — your own work, because the exercise is about your habits** rather than about somebody else's code.
- **The keyboard-only pass first** (Module 05 §2), with every unreachable control listed.
- **A screen reader pass** (Module 03 §2), recording what is announced for each control and what is silent.
- **The contrast and colour-vision checks** (Module 03 §3), with the failures measured rather than judged.
- **An automated checker run, and the defects it missed that you found manually, counted.**
- **And each defect classified by the capability it excludes** rather than by the criterion number, which is what makes the list actionable.

Done means

- **The manual findings exceeding the automated ones — which they will, substantially**, and the ratio is the point of the exercise (Module 09 §1).
- **Every defect tied to a capability**, so that the repair is a design decision rather than a compliance edit.
- **The screen reader transcript included**, because **what is actually announced is frequently not what you assumed**.
- **And the honest total. A first audit of your own work finds more than you expect**, and reporting that accurately is the assessed behaviour.

Project 2 — Repair it, and prove the repair (after Module 12)

Fix what you found, test it with the technology, and state what remains.

Requirements

- **The defects from Project 1 repaired**, with **each repair justified from the mechanism** (Module 08) rather than from a rule.
- **Native elements preferred over ARIA** wherever possible, and **every ARIA use justified** (Module 08 §3's first rule).
- **Re-tested with the actual assistive technology**, not only with the checker — **and the transcripts before and after**.
- **The 400% zoom and reflow case handled**, which is where most layouts fail.
- **One non-visual representation of a figure or data** (Module 10), which is the hardest requirement here.
- **And a remaining-defects list that is specific and non-empty**, in Module 13 §2's form.

Done means

- **The repairs justified from the accessibility tree rather than from the criterion — which is the difference between understanding this and passing it** (Module 08).
- **Tested with the technology**, because **a checker pass is not evidence that anybody can use it** (Module 09 §1).
- **The remaining-defects list non-empty and specific — a claim of full accessibility fails this project**, for the reasons Module 13 §1 gives.
- **And at least one repair that improved the interface for everybody**, named as such. **That is the course's central claim**, and demonstrating it once on your own code is what makes it believable.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
The W3C WAI materials, including the tutorials and Understanding WCAG (free)	The primary source for Modules 03 through 08 , free in full. The Understanding documents give the reason behind each criterion, which is the part worth reading — the criteria themselves are terse by design.
WCAG 2.2, with the quick reference (free)	Module 07's object of study . Read the structure first — principles, guidelines, criteria, levels — because the structure is what makes the document navigable.
The ARIA Authoring Practices Guide (free)	Module 08 . Patterns with their keyboard interactions and their markup, and the first rule of ARIA is stated early and should be believed.
Microsoft's Inclusive Design toolkit (free)	Module 02's permanent, temporary, and situational framing comes from here, and it is the most persuasive single page in the subject.

Resource	Role
The Game Accessibility Guidelines (free)	Module 11. Organised by effort rather than by disability, which makes it unusually usable mid-project.
Lazar, Goldstein & Taylor — Ensuring Digital Accessibility through Process and Policy	Module 12. Why accessibility fails for organisational rather than technical reasons, which is the part engineering courses omit. Library copy.
The WebAIM Million annual report (free)	Module 09's measurement. An automated audit of a million home pages, repeated yearly — and the numbers are worse than anybody expects and are not improving quickly.

Tooling

- **A screen reader, and you have to learn to use it — NVDA on Windows, VoiceOver on macOS and iOS, TalkBack on Android,** all free or built in. **Module 03 §2 is where you learn the navigation model,** and an hour spent there changes how you write markup permanently.
- **Your keyboard, with the mouse unplugged** — which is **the cheapest accessibility test that exists** and catches a large fraction of real defects (Module 05 §2).
- **An automated checker — axe, WAVE, or Lighthouse** — and **Module 09 §1 reports what fraction of criteria these can check at all,** which is the number to know before trusting a green result.
- **A contrast checker and a colour-vision simulator,** run on every figure and every interface (Modules 03 §3 and CSCE 679 Module 04).
- **Operating system magnification, at 400%, and browser zoom at 400% with reflow** — which breaks more layouts than anything else on this list.
- **And, where you can arrange it, somebody who uses assistive technology daily — which finds a class of problem no checklist contains,** and Module 09 §4 is about arranging that properly and paying for it.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.